Veterinary Database Program

Task: Written assignment

Program: LIBFWAWADLMDSPWP01 – Programming with Python

Date: 12/03/2025

Author: Harvey Liam Plaxton

Matriculation no. 10242306

Lecturer: Thomas Kopsch

1. Introduction

Database reliant programs are widely used in a variety of modern industries in nearly all sectors ranging from healthcare and education to business and logistics (Kumar et al., (2020)). Database systems provide both secure and organised storage of critical information concisely and effectively (Ochs et al., (2019)). The programs built around them are such that they allow users to interact with the data stored. Thus, it is crucial that these programs be efficient and user-friendly with simple UI and functionality to allow for seamless data retrieval (Figueiredo et al., (2017)). They must prioritise succinct forms to append new data to the dataset, and overall must maximise the availability of user manipulation of data without risking the security of the information stored on the database.

Medical care systems are reliant on the accessibility of existing data from past records (such as patient histories, ongoing treatments, care plans, etc) when providing treatment and diagnosis (Li et al., (2021)), and hence can greatly benefit from the use of database programs (Chaudhry et al., (2006)). Among the vast areas of the medical sector, veterinary care, in particular, presents unique challenges that can benefit from specialized database programs. Hence, my inspiration for this program; I specifically chose veterinary care as the tangible concept to build this program around. It is worthy of noting, however, that the program is not limited to the particular use of veterinary care, as I will discuss later on.

I chose veterinary care in particular due to its lesser priority in the modern medical world (Lustgarten et al., (2020)), the relevant need for database systems in the field (Houe et al., (2011)), and due to its uniqueness in field requirements. Unlike human patients, patients in veterinary care are often identified by a combination of multiple factors/forms of identification; patient name, the name of their owner, potential chip number, etc. This multifaceted identification allowed me to incorporate various forms of record retrieval in the program. This allows for more a complex query system rather than just a trivial name search, and thus is more relevant to real world applications.

The choice to build a database program around veterinary care also reflects the need for specialized tools in niche fields as well as the modern need to expand veterinary data (Cockcroft and Holmes, (2008)). Veterinary clinics often require quick access to patient records (Toews et al., (2011)). Such records must be organized and accessible at all times to ensure efficient operation. This inspires the need for the inclusion of fields like patient condition and arrival date and the method in which they are appended and retrieved. These fields will be discussed further in the text.

The aim of this project is to create a simple, user-friendly program that is highly modifiable and expandable, with basic yet relevant fields that offer insight into the utility of the program. The following text will describe the classes and functions used to construct the program, the packages used to facilitate the process, insights into the development process, possible adaptations of the program, and the reasoning behind the decisions taken throughout. The text will be concluded with

some final thoughts regarding specific areas of the program. As a further note, the program in its entirety can be found in Appendix A at the end of the text.

2. Packages

Three Packages were used to build the program. The first and most integral package used is the Pandas library. Specifically, the Pandas library was chosen to build the data storage system through the use of Pandas dataframes. Pandas dataframes ensure the data is stored in an organised, tabular structure consisting of ordered rows and columns, which improves data handling and readability when viewing the entire dataset. Furthermore, the Pandas library is complete with built-in methods of data-handling and modification, and also is optimised for handling large quantities of data (Mckinney, (2011)). The prior being critical in veterinary systems that must handle vast amounts of patient data. Pandas also allows for simplified methods of data exporting. Lastly, dataframes are integrated seamlessly with other visualisation packages such as Bokeh, which is the second package used in the program.

The functions *figure*, *show*, and *output_file* from the Bokeh.io and Bokeh.plotting libraries are used in the program. Using Bokeh to visualise plots allows for high-quality, interactive, web-based plots that are both saved to the users' desktop and can be easily customised. Bokeh's server-side capabilities also allow it to handle large dataframes efficiently. Bokeh's compatibility with Pandas' dataframes permits the "output_file" function to save the dataset as a csv seamlessly.

The final package used in the program is the Datetime package. The datetime package ensures accurate retrieval of current date data as well as the ease of reformatting the date into a suitable form for users.

3. Program Breakdown

The "Patient" class serves as the foundation for the program. It functions for the creation of the "Patient" objects that store key information about each patient. This includes the fields "patient_id", "patient_name", "owner_name", "condition", and "arrival_date". These fields were chosen to provide a basic understanding of the program's capabilities, while not overcrowding the program for testing purposes. Three separate naming fields are included to represent the unique complexity of veterinary care patients; patients generally have a name as well as the name of their caretakers. The final naming field "patient_id" is used to provide a unique id for all patient objects in the dataframe.

Followed by this, the "Database" class is initialised through inheriting the patient object. The database class creates a Pandas dataframe under the column headings matching that of the input fields from the Patients class; "Patient ID", "Patient Name", "Owner Name", "Condition", "Date Arrival". The database class contains key functions for creating and manipulating patient data; "add_patient", "visualize_conditions", "find_patient_by_id", "find_patient_by_name",

“find_patient_by_owner”, “delete_patient”, “edit_patient”, “list patients”, and “export_to_csv”. These functions were chosen to represent the particular needs of a veterinary care system.

The “add_patient” function inherits the patient object and extracts its’ attributes to form a dictionary which is then appended to the dataframe. In particular, the function uses the *pandas.concat* method to append new patient data, which ensures the system is optimised as the dataframe and function are both part of the same library.

A common feature of medical systems is to visualise patient statistics; thus the “visualize_conditions” function produces an example bar chart of the totals in each patient condition category; “mild”, “moderate”, “severe” (Figure 1). This is done using the Bokeh library, in particular the “vbar” method which is then exported as a HTML file. This method of output ensures the graph is accessible on most platforms and makes use of Bokeh’s interactive graph features. Firstly, the function ensures the database is not empty via an if statement, in which case the program is restarted and the phrase “No patients in the database to visualize” is printed.

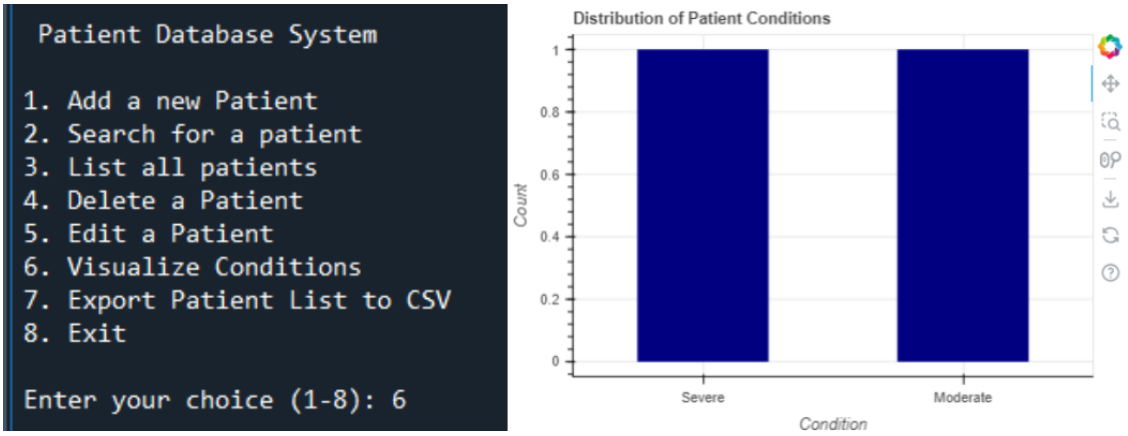


Figure 1. Program option 6; Visualize Conditions alongside its graph output in default web browser.

The “find_patient_by_id”, “find_patient_by_name”, and “find_patient_by_owner” functions search the database for matching patient records by use of either the “patient_id”, “patient_name” or “owner_name” fields. In the case that no matching record is found the function returns None. Using the string-matching method also allows the program to search using “case = False”, this way case errors in user input can be bypassed. In the case of searching by patient_id, the system will produce only one matching record as the id is unique for all patients. In the case of both the searching by patient name or patient owner functions, the function returns a list of all matching records. This is essential for scenarios where patient id is unknown. Allowing for various search methods improves user control and understanding. It also represents the complexity of record retrieval in medical care systems.

The “delete_patient” function is also essential to the program as it functions as the method of deleting existing patients. This would be required when patients are no longer in care. The function requires a “patient_id” as input, with which it employs Boolean indexing to filter out the patient with

the specified ID, effectively removing the record from the dataframe. Patient ID must be used to locate a matching record as it is a unique field and ensures a single record will be deleted. Further in the program a confirmation step is required (seen in the “remove_patient” function).

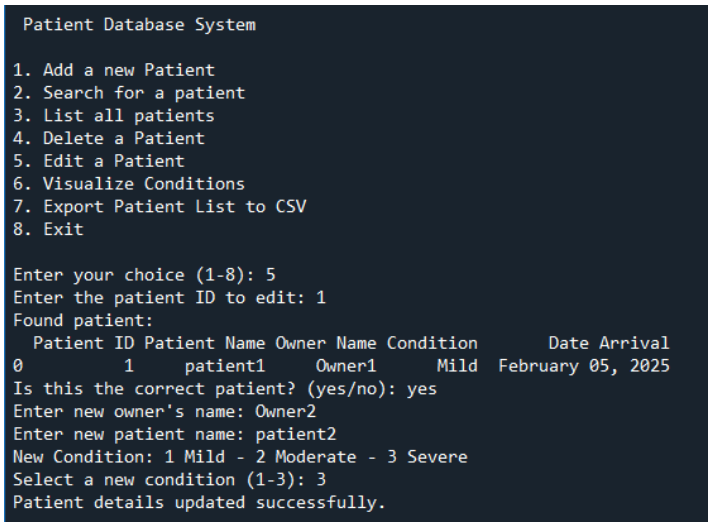


Figure 2. Editing existing patient record.

To improve data manipulation, users can not only add and delete patients, but also edit their records. The “edit_patient” method is an interactive function that allows users to modify a patient's information. It first retrieves the patient record using the prior “find_patient_by_id” function. It then displays the details of the record retrieved and requests confirmation. After confirming the selection, it prompts the user to input new data for fields “owner_name”, “patient_name”, and “condition” (Figure 2). The function then updates the dataframe using pandas' “loc” method, ensuring the changes are applied directly to the corresponding record. If incorrect inputs are detected (i.e. selecting an option not in the menu list) then the system reject the input with a suitable warning message and resets to the starting menu. Editing records is essential for maintaining accurate and up-to-date records in any database system, and is crucial even more so in medical care systems.

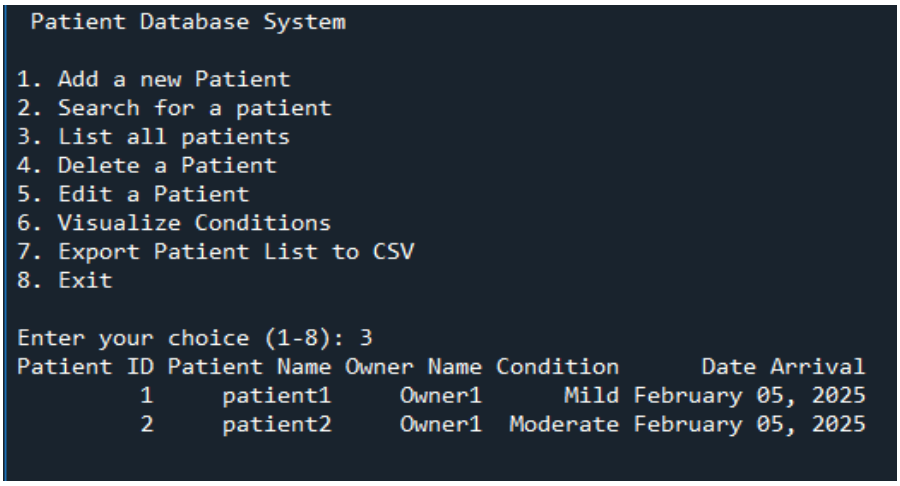
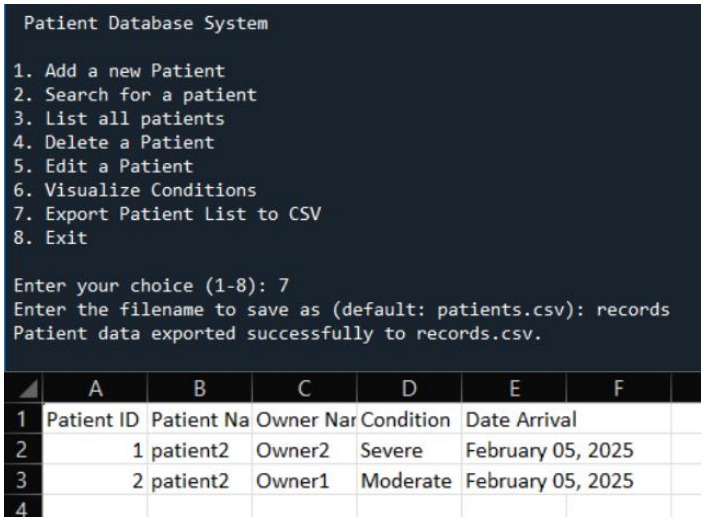


Figure 3. Listing all existing patients in the database

The “list_patients” function prints all the records in the database in a tabular format using the “to_string” method from pandas (Figure 3). This feature is vital for providing users with an overview of the current database state, allowing them to see all registered patients at once. The method includes a check to ensure the database is not empty before attempting to print, hence handling potential edge cases. By offering a user-friendly way to display data, this function improves the program's utility and aligns with the principle of making data accessible to its users.



	A	B	C	D	E	F
1	Patient ID	Patient Na	Owner Nar	Condition	Date Arrival	
2	1	patient2	Owner2	Severe	February 05, 2025	
3	2	patient2	Owner1	Moderate	February 05, 2025	
4						

Figure 4. Top: Exporting patients database using program option 7. Bottom: Exported csv file open in Microsoft Excel.

Finally, the “export_to_csv” method provides functionality to save the current state of the database to a CSV file (Figure 4). This function uses the “to_csv” method from pandas to convert the dataframe to a file. The method checks if the database is empty to prevent exporting invalid data and adds a .csv extension if it's missing from the provided filename (file name is requested in the menu options). This feature is essential for creating backups, sharing data, or integrating with other systems.

The next section of the program follows the connection between the user and the dataframe. It covers functions that create the menu allowing the user to interact via option selection and user-friendly navigation.

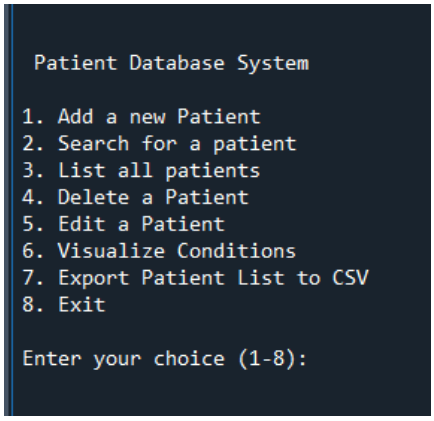


Figure 5. Interactive program menu.

Firstly, the function “display menu” is the method responsible for presenting the user with a menu of available options and capturing their selection. By utilizing Python's print and input functions, this method ensures a clear and user-friendly interface (Figure 5). The numbered menu options will map directly to key operations of the program, such as adding, searching, or deleting patients.

The “add_new_patient” function (Figure 6) is essential for creating new entries in the veterinary care database. It begins by determining the next available patient_id by checking the current maximum ID in the database or assigning "1" if the database is empty. This approach ensures each patient has a unique identifier. The function then interacts with the user via input prompts to gather patient details; owner's name, patient's name, and condition. A conditional structure maps the user's condition choice to the condition options ("Mild," "Moderate," "Severe"). The program provides suitable feedback if an input is not invalid and returns to the starting menu. The function also uses the “date” module from the “datetime” library to capture the current date, and then a Patient object is instantiated with the collected data using the Patient class. This object is then passed to the “add_patient” method of the Database class to finalise the addition of the patient to the dataframe.

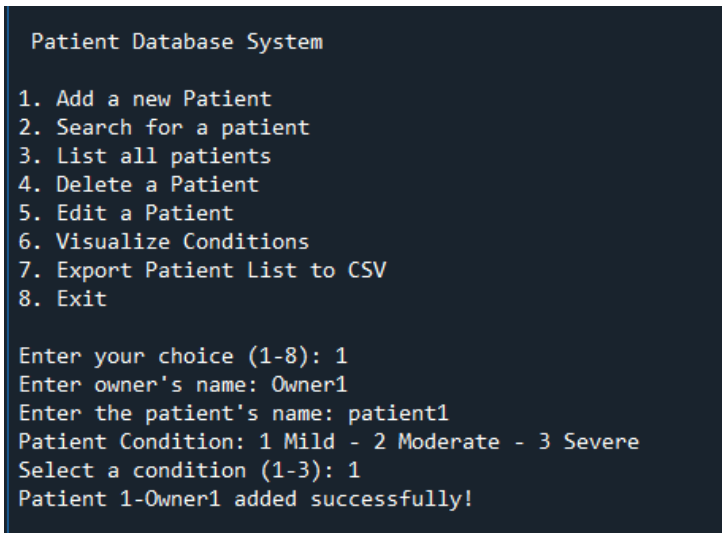


Figure 6. Program use for entering new patient record with example inputs (program option 1). Example owner name: Owner1, example patient name: patient1, example condition: 1.

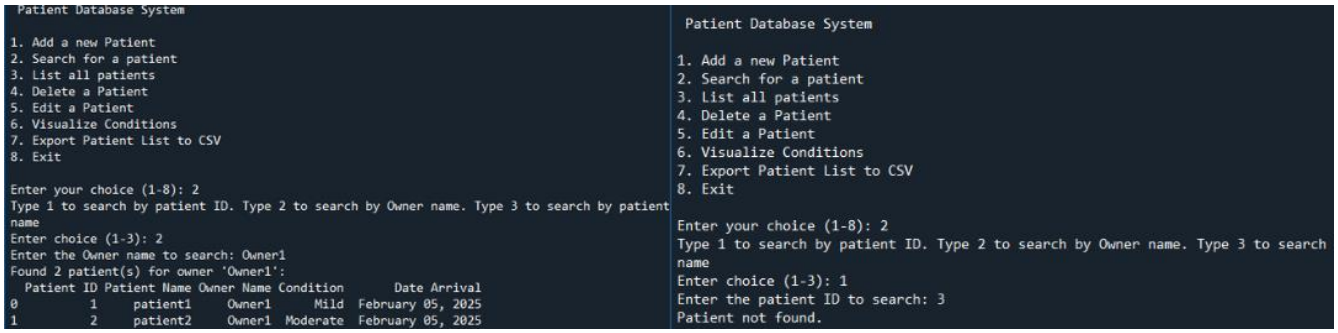


Figure 7. Left: Searching for a record using the search by owner name option. Right: Searching for a record not in the database with program user feedback.

The “search_patient” function (Figure 7) provides a versatile interface for locating patient records in the database. It first prompts the user to select a search method; “patient_id”, “owner_name”, or “patient_name”. Each search option is then delegated to specific methods (find_patient_by_id, find_patient_by_owner, and find_patient_by_name) within the Database class. For each search type, the function captures user input, queries the database, and displays the results. If no match is found, an appropriate message is printed to inform the user.

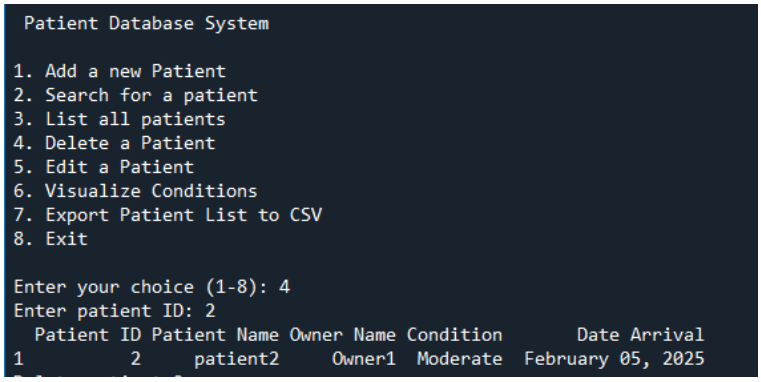


Figure 8. Removing a patient record from the database (program option 4) by first searching for the desired record followed by confirmation for deletion.

Furthermore, the “remove_patient” function allows users to delete specific patient records from the database (Figure 8). It begins by prompting the user for a patient_id (as the patient ID is unique), which it uses to locate the corresponding patient via the “find_patient_by_id” method within the database class. If a matching record is

found, the function displays the patient details and asks the user for confirmation before proceeding with deletion. The use of confirmation ensures that users cannot accidentally delete records. The function then calls the “delete_patient” method from the Database class to remove the record. If the specified “patient_id” is not found, the program informs the user, fulfilling the need for error handling and user feedback as key components of the program's design.

The final section of the program serves as the main execution block, ensuring that the veterinary care system operates effectively and interactively. It begins by instantiating an object of the Database class, which acts as the main structure for managing patient data. The program enters a continuous loop that starts by calling the “display_menu” function. Each menu option corresponds to a specific function that is triggered based on the user’s input. The use of conditional statements ensures that the program executes the correct functionality based on the user's choice. It also incorporates safeguards by using input validation; if the user inputs a choice not within 1-8 then the program returns a warning message and returns to the menu. Option 8 also includes the option to terminate the program, in which case the program stops running via a break in the continuous loop.

4. Program Use

Upon running the program, users are prompted by an 8-option menu to manually enter a digit (1-8) associated with the desired option. The program functions in a continuous loop until that loop is closed, which can be done by selecting option 8 to exit the program. The program stores data in a dataframe until the program is closed, at which point the database data is cleared.

Selecting an option further prompts the user according to the option selected. Thus, selecting "Add a new Patient" by entering “1” in the input prompt calls the “add_new_patient” function, which facilitates the addition of a new Patient object to the database. In the instance that the options selected are beyond the given range, the program sends the user back to the starting menu. This form of error response occurs in all areas of the program for incorrectly entered data. In most cases the program responds with user feedback indicating the error in the data entered.

Users can then manipulate the entered data according to their needs by using the presented menu options; data deletion, editing, and exporting.

5. Personal Evaluation

The overall program came with many challenges during its’ inception. One of these was storing the data in a way such that data manipulation could be handled efficiently. Originally, patient data was stored in a list, which meant retrieving and manipulating data (particularly in the case of editing existing data) became tedious and slow. This problem was averted by converting to Pandas dataframes. As previously mentioned, these dataframes provide optimised methods of data retrieval allowing for faster and more secure options for data manipulation in the program.

Furthermore, creating a concrete, unique ID value for each patient was challenging in the case of deleting patients and not overlapping existing ID values. To overcome this, I used a simple indexing system. The program would first check for existing entries and if none were found then it begins the indexing at 1. If exiting entries are found then the program identifies the maximum index value and increments this value by 1 to assign the new index for the newly added entry.

The most intriguing aspect of building the program was prioritising user interactivity and designing ways of providing user feedback, especially within the navigation of the menu. In addition to this, I particularly enjoyed developing with the intention of making it as user proof as possible. From this pursuit the concept for using the datetime package to extract the date rather than the user manually inputting the date arose. This also led to the development choice of using a selection system for patient condition severity and patient indexing. Using a numbering system for condition severity allows the program to avoid user error in misspellings and also reduces input time, which is essential for medical based systems.

For future development, this program could be extended to include a wider range of applicable fields such as treatment plan, diagnosis, a list of treatment times, an editable notes field, etc. Furthermore, an additional status field to indicate the success of the treatment and thus success charts could be produced to analyse the overall effectiveness of various treatment plans (this can be done by expanding upon the “visualize_conditions” function). Furthermore, a feature to allow users to attach files relevant to a specific patient may be helpful for cases such as uploading a patient’s test results, images of scans, etc.

6. Conclusion

In conclusion, the development of this veterinary database program prioritises the importance of proper data management in a medical database. By utilizing Pandas for structured data handling, Bokeh for visualization, and Datetime for accuracy, alongside the various data management options, the program provides the foundational functions required for an operational database system. Although simple, the modular approach to the program ensures flexibility. This allows for ease of data retrieval, modification, and deletion, while also prioritizing user interactivity and thus enhancing accessibility.

Beyond veterinary care, the program’s overall structure can be adapted to various medical and non-medical applications. Potential future improvements, such as incorporating additional patient details, treatment tracking, and file attachment capabilities, could further enhance its capability. Ultimately, the experience of developing this program provided valuable insights into database design.

References

Chaudhry, B., Wang, J., Wu, S., Maglione, M., Mojica, W., Roth, E., Morton, S.C. and Shekelle, P.G. (2006). Systematic review: impact of health information technology on quality, efficiency, and costs of medical care. Annals of internal medicine, 144(10), pp.742–52.

Cockcroft, P. and Holmes, M., 2008. Handbook of evidence-based veterinary medicine. John Wiley & Sons.

Figueiredo, M.S.N. and Pereira, A.M. (2017). Managing Knowledge – The Importance of Databases in the Scientific Production. Procedia Manufacturing, 12, pp.166–173.

Houe, H., Egenvall, A., Virtala, A.-M., Olafsson, T. and Østerås, O. (2011). Databases in veterinary medicine – validation, harmonisation and application: introduction. Acta Veterinaria Scandinavica, 53(S1).

Kumar, R. and Prasad, P. (2020). Impact of database management in modern world. Available at: https://www.researchgate.net/publication/343323880_Impact_of_database_management_in_modern_world.

Li, L., Novillo-Ortiz, D., Azzopardi-Muscat, N. and Kostkova, P. (2021). Digital Data Sources and Their Impact on People's Health: A Systematic Review of Systematic Reviews. Frontiers in Public Health, 9.

Lustgarten, J.L., Zehnder, A., Shipman, W., Gancher, E. and Webb, T.L. (2020). Veterinary informatics: forging the future between veterinary medicine, human medicine, and One Health initiatives—a joint paper by the Association for Veterinary Informatics (AVI) and the CTSA One Health Alliance (COHA). JAMIA Open, 3(2), pp.306–317.

Mckinney, W. (2011). pandas: a Foundational Python Library for Data Analysis and Statistics. ResearchGate. Available at: https://www.researchgate.net/publication/265194455_pandas_a_Foundational_Python_Library_for_Data_Analysis_and_Statistics

Ochs, A.R., Mehrabi, M., Becker, D., Asad, M.N., Zhao, J., Zaragoza, M.V. and Grosberg, A. (2019). Databases to Efficiently Manage Medium Sized, Low Velocity, Multidimensional Data in Tissue Engineering. Journal of Visualized Experiments, (153).

Toews, L. (2011). The Information Infrastructure that Supports Evidence-Based Veterinary Medicine: A Comparison with Human Medicine. Journal of Veterinary Medical Education, 38(2), pp.123–134.

Appendix A – Python program

```
# Import the required packages

from datetime import date

import pandas as pd

from bokeh.plotting import figure, show

from bokeh.io import output_file
```

```
#%% Create the Patient Class

class Patient:

    def __init__(self, patient_id, patient_name, owner_name, condition, date_arrival):

        #Setting the variables

        self.patient_id = patient_id

        self.patient_name = patient_name

        self.owner_name = owner_name

        self.condition = condition

        self.date_arrival = date_arrival

#Produce the layout for printing a Patient record.

    def __str__(self):

        return(f"Patient ID: {self.patient_id}, Name: {self.patient_name}, Owner: {self.owner_name}\n"

               f"Condition: {self.condition}, Arrival Date: {self.date_arrival}")

#%%


# Create the Database Class

class Database:

    def __init__(self):

        # Initialize the database as a pandas DataFrame

        self.patients = pd.DataFrame(columns=["Patient ID", "Patient Name", "Owner Name", "Condition", "Date Arrival"])


    def add_patient(self, patient):

        # Append a new patient as a dictionary

        new_patient = {

            "Patient ID": patient.patient_id,

            "Patient Name": patient.patient_name,

            "Owner Name": patient.owner_name,

            "Condition": patient.condition,

            "Date Arrival": patient.date_arrival

        }

        self.patients = pd.concat([self.patients, pd.DataFrame([new_patient])], ignore_index=True)
```

```
def visualize_conditions(self):
    """Visualize the distribution of conditions using Bokeh."""
    if self.patients.empty:
        print("No patients in the database to visualize.")
        return

    # Count the occurrences of each condition
    condition_counts = self.patients["Condition"].value_counts()

    # Create a Bokeh bar chart
    p = figure(
        title="Distribution of Patient Conditions",
        x_range=condition_counts.index.tolist(),
        x_axis_label="Condition",
        y_axis_label="Count",
        height=350,
        width=500
        )
    p.vbar(x=condition_counts.index, top=condition_counts.values, width=0.5, color="navy")

    # Specify an output file and show the plot
    output_file("conditions_distribution.html")  # Output will be saved to this file
    show(p)

#Define functions for record searching

def find_patient_by_id(self, patient_id):
    #Search for matching ID and return the list
    result = self.patients[self.patients["Patient ID"] == patient_id]
    return result if not result.empty else None
```

```
def find_patient_by_name(self, patient_name):
    #Search for matching patient name and return the list
    result = self.patients[self.patients["Patient Name"].str.contains(patient_name, case=False, na=False)]
    return result if not result.empty else None

def find_patient_by_owner(self, owner_name):
    #Search for matching owner anme and return the list
    result = self.patients[self.patients["Owner Name"].str.contains(owner_name, case=False, na=False)]
    return result if not result.empty else None

# Define function for deleting a record
def delete_patient(self, patient_id):
    #Exclude the matching record from the patients list
    self.patients = self.patients[self.patients["Patient ID"] != patient_id]

#Define the function for modifying a record
def edit_patient(self, patient_id):
    #Find the record with matching ID
    patient = self.find_patient_by_id(patient_id)
    #Produce user feedback for the record
    if patient is not None:
        print(f"Found patient:\n{patient}")
        #Request confirmation
        confirmation = input("Is this the correct patient? (yes/no): ").lower()
        # Display fields that can be modified
        if confirmation == 'yes':
            new_owner_name = input("Enter new owner's name: ")
            new_patient_name = input("Enter new patient name: ")
            print("New Condition: 1 Mild - 2 Moderate - 3 Severe")
            condition_choice = input("Select a new condition (1-3): ")
            if condition_choice == '1':
                condition = 'Mild'
```

```
            elif condition_choice == '2':
                condition = 'Moderate'
            elif condition_choice == '3':
                condition = 'Severe'
            #Provide user feedback on incorrect inputs
            else:
                print("Invalid choice, no changes made to condition.")
                return
            #Modify the record with new inputs
            self.patients.loc[self.patients["Patient ID"] == patient_id, ["Owner Name", "Patient Name", "Condition"]] = [
                new_owner_name, new_patient_name, condition]
            #User feedback for successful update
            print("Patient details updated successfully.")
        else:
            #User feedback for unsuccessful update
            print("Edit operation cancelled.")
    else:
        print("Patient not found.")


#Define function for returning the database
def list_patients(self):
    if not self.patients.empty:
        print(self.patients.to_string(index=False))
    else:
        #Provide feedback for an empty database
        print("No patients in the database.")


#Define function to export as csv file. Apply a default if no filename is entered.
def export_to_csv(self, filename="patients.csv"):
    """Export the list of patients to a CSV file."""
    #Cancel operation for an empty database
    if self.patients.empty:
```

```
            print("No patients in the database to export.")
            return

        # Ensure the file has a .csv extension
        if not filename.endswith(".csv"):
            filename += ".csv"

        #Export and provide user feedback
        self.patients.to_csv(filename, index=False)
        print(f"Patient data exported successfully to {filename}.")
#%%

# Function to display the menu
def display_menu():
    print("\n")
    print("\n Patient Database System \n")
    print("1. Add a new Patient")
    print("2. Search for a patient")
    print("3. List all patients")
    print("4. Delete a Patient")
    print("5. Edit a Patient")
    print("6. Visualize Conditions")
    print("7. Export Patient List to CSV")
    print("8. Exit \n")
    choice = input("Enter your choice (1-8): ")
    return choice
#%%

# Add a new patient
def add_new_patient(database):
    # Provide indexing for the new record
    if not database.patients.empty:
```

```
        patient_id = str(int(database.patients["Patient ID"].max()) + 1)
    else:
        patient_id = "1"

    #Display user inputs for required fields
    owner_name = input("Enter owner's name: ")
    patient_name = input("Enter the patient's name: ")
    print("Patient Condition: 1 Mild - 2 Moderate - 3 Severe")
    condition_choice = input("Select a condition (1-3): ")
    if condition_choice == '1':
        condition = 'Mild'
    elif condition_choice == '2':
        condition = 'Moderate'
    elif condition_choice == '3':
        condition = 'Severe'
    else:
        print("Invalid choice, please select option 1, 2, or 3.")
        return

    #Retrieve the date using the datetime library
    today = date.today()
    date_arrival = today.strftime('%B %d, %Y')
    # Create and add the record using the obtained inputs
    new_patient = Patient(patient_id, patient_name, owner_name, condition, date_arrival)
    database.add_patient(new_patient)
    print(f"Patient {patient_id}-{owner_name} added successfully!")
#%%

# Search for a patient
def search_patient(database):
    #Produce options for the various searchign methods and return the relevant choice
    print("Type 1 to search by patient ID. Type 2 to search by Owner name. Type 3 to search by patient name")
```

```
    search_choice = input("Enter choice (1-3): ")
    if search_choice == '1':
        patient_id = input("Enter the patient ID to search: ")
        patient = database.find_patient_by_id(patient_id)
        if patient is not None:
            print(patient)
        else:
            print("Patient not found.")
    elif search_choice == '2':
        owner_name = input("Enter the Owner name to search: ")
        patients = database.find_patient_by_owner(owner_name)
        if patients is not None:
            print(f"Found {len(patients)} patient(s) for owner '{owner_name}':")
            print(patients)
        else:
            print("Patient not found. Beware of case sensitivity when searching.")
    elif search_choice == '3':
        patient_name = input("Enter the patient's name: ")
        patients = database.find_patient_by_name(patient_name)
        if patients is not None:
            print(f"Found {len(patients)} patient(s) under the name '{patient_name}':")
            print(patients)
        else:
            print("Patient not found. Beware of case sensitivity when searching.")
    #Provide feedback for an incorrect selection
    else:
        print("Invalid Choice, please choose option 1, 2, or 3.")
#%%


# Delete a patient
def remove_patient(database):
    #Request a patient ID
```

```
    patient_id = input("Enter patient ID: ")
    patient = database.find_patient_by_id(patient_id)
    if patient is not None:
        print(patient)
        print(f"Delete patient {patient_id}")
        #Request confirmation for deletion
        confirmation = input("Type 1 to confirm: ")
        if confirmation == '1':
            database.delete_patient(patient_id)
            print(f"Patient {patient_id} has been successfully deleted")
        else:
            print("Operation cancelled")
    #Provide feeback when a matching record is not obtained
    else:
        print("Patient not found.")
#%%


# Main program
if __name__ == "__main__":
    db = Database()


    while True:
        # Produce the interactive menu
        choice = display_menu()


        # Run functions associated with user input
        if choice == '1':
            add_new_patient(db)
        elif choice == '2':
            search_patient(db)
        elif choice == '3':
            db.list_patients()
```

```
elif choice == '4':
    remove_patient(db)
elif choice == '5':
    patient_id = input("Enter the patient ID to edit: ")
    db.edit_patient(patient_id)
elif choice == '6':
    db.visualize_conditions()
elif choice == '7':
    #Request an optional filename
    filename = input("Enter the filename to save as (default: patients.csv): ").strip()
    if not filename:
        filename = "patients.csv"
    db.export_to_csv(filename)
#Option to end the program
elif choice == '8':
    print("Closing Program")
    break
else:
    print("Invalid choice. Please select a valid option.")
```

End of paper